



UNIVERSITÀ  
DEGLI STUDI DI BARI  
ALDO MORO

MASTER DEGREE IN COMPUTER SCIENCE

URBAN SECURITY

---

## Local RAG System

Autore: Leonardo Moro

Matricola: 869146

### Summary

|                                                                 |   |
|-----------------------------------------------------------------|---|
| 1. Introduction.....                                            | 2 |
| 1.1 Project Scope: Application to UAV Manuals.....              | 2 |
| 2. Research .....                                               | 3 |
| 2.1 The Mechanics of Retrieval-Augmented Generation (RAG) ..... | 3 |
| 2.1.1 Chunking Strategies .....                                 | 3 |
| 2.1.2 Embeddings.....                                           | 3 |
| 2.1.3 Vector Databases.....                                     | 4 |
| 2.2 Prompt Engineering Techniques .....                         | 4 |
| 3. System Implementation & Architecture.....                    | 4 |
| 3.1 Data Ingestion and Processing Pipeline.....                 | 5 |
| 3.2 Vector Storage and Semantic Search .....                    | 5 |
| 3.3 Inference Core and Prompt Alignment .....                   | 6 |
| 4. Automated Testing & Evaluation Framework.....                | 7 |
| 5. Test Results & Evaluation Analysis .....                     | 8 |
| 5.1 In-Scope Query Evaluation .....                             | 9 |
| 5.2 Out-of-Scope Query Evaluation & Hallucination Block.....    | 9 |
| 6. Conclusion & Future Works .....                              | 9 |

## 1. Introduction

Large Language Models (LLMs) are highly capable of understanding and generating natural language. However, when applied to technical and specialized domains, standard LLMs face two main limitations:

1. **Knowledge Cutoff:** The model only knows the information present in its training dataset and cannot access private or specific documents.
2. **Hallucinations:** The model can generate incorrect or fabricated information with high confidence, which is dangerous in engineering contexts.

To resolve these limitations without expensive fine-tuning, this project implements a Retrieval-Augmented Generation (RAG) architecture.

A RAG system can be described as an "open-book" approach for AI. Instead of answering questions relying only on its static memory, the system follows three sequential steps:

1. **Retrieval:** When a user asks a question, the system searches an external knowledge base (in this case, a processed PDF document) and extracts the most relevant text segments.
2. **Augmentation:** The system combines the user's original question with the extracted text segments into a single, enriched prompt.
3. **Generation:** The prompt is sent to the LLM. The model reads the provided context and generates an answer that is strictly grounded in the document.

### 1.1 Project Scope: Application to UAV Manuals

In this project, the RAG framework is applied to the domain of **Unmanned Aerial Vehicles (UAVs)**, specifically utilizing the official technical manual of the DJI Mini 2 drone.

Operating a UAV requires strict adherence to safety protocols, battery limitations, and emergency procedures. In real-time or critical flight scenarios, a pilot cannot waste time searching through a hundred-page PDF document to find specific technical parameters, such as maximum wind resistance or sensor calibration steps.

This project addresses this issue by creating a local, terminal-based assistant. By indexing the DJI Mini 2 manual into a local vector database, the operator can query the document using natural language and receive immediate answers.

The primary objectives of this implementation are:

**Data Accuracy:** The system must accurately extract complex data, including technical tables and multi-step safety checklists, without truncating the information.

**Source Traceability:** The system is configured to append the exact source (`chunk_number`) to every answer. This allows the operator to verify the information directly in the official manual, ensuring safety and transparency.

## 2. Research

### 2.1 The Mechanics of Retrieval-Augmented Generation (RAG)

To build an isolated and reliable assistant, it is fundamental to understand the three core components that define a local RAG pipeline: chunking strategies, embeddings, and vector databases.

#### 2.1.1 Chunking Strategies

A standard Large Language Model has a maximum limit of text it can read at one time (known as the context window). Furthermore, if you pass an entire 100-page manual to the model, the specific technical data can get lost in the middle of the text. For this reason, we use **chunking**, which means breaking down the large PDF document into smaller, manageable blocks of text.

In technical manuals, choosing the right chunking strategy is critical:

If a chunk is too small, it loses the general meaning of the paragraph.

If a chunk is too large, the specific technical numbers (like weights or frequencies) get diluted and hard to find.

To solve this, the community uses advanced splitters like the **RecursiveCharacterTextSplitter**. This tool does not cut the text blindly. Instead, it looks for logical separators like double newlines (`\n\n`), single newlines (`\n`), and spaces (). This ensures that sentences and technical tables stay together as much as possible. Additionally, an overlap is configured between chunks, meaning that the end of Chunk 1 is repeated at the beginning of Chunk 2. This technique prevents losing information that sits exactly on the border of a page split.

#### 2.1.2 Embeddings

An AI model does not understand text the way humans do; it understands math.

**Embeddings** are the technology that translates words and sentences into a long list of numbers, called a vector.

When a text block is transformed into an embedding, it is placed inside a multi-dimensional mathematical space. The key feature of embeddings is semantic similarity: texts that share similar meanings are placed close to each other in this mathematical space. For example, the vector for the word **"battery"** will be closer to the vector for **"charging"** than to the vector for **"propeller"**. This allows the system to find relevant pages in the manual even if the user uses slightly different words than the official text.

### 2.1.3 Vector Databases

A traditional database searches for data using exact keywords. If you type "**flight limits**", it will only find rows containing that exact phrase. A Vector Database, instead, is designed specifically to store and index the mathematical vectors generated by the embedding model.

When the user asks a question, the system converts the question into a vector and uses a mathematical formula (such as Cosine Similarity) to calculate the distance between the question vector and all the manual chunk vectors. The database then instantly returns the top K most relevant chunks (the closest ones in the vector space), regardless of exact keyword matching. This process forms the core of the **Retrieval phase**.

## 2.2 Prompt Engineering Techniques

Once the vector database retrieves the correct text blocks from the manual, they must be presented to the LLM. **Prompt Engineering** is the practice of designing precise text instructions to control and align the behavior of the AI model.

In an engineering project, standard open-ended prompts are insufficient because they allow the model to rely on its pre-trained memory, leading to hallucinations. To enforce strict reliability, specific guardrails must be injected into the prompt:

1. **Context Anchoring**: The prompt must explicitly command the LLM to act as a strict context processor. It must state that the answer must be derived only and exclusively from the provided text blocks.
2. **Strict Refusal Clause**: To eliminate hallucinations when a user asks an out-of-scope question (e.g., a cooking recipe), the prompt must include a fallback instruction forcing the model to say a standardized refusal phrase, such as "Information not found in the manual", rather than inventing an answer.
3. **Deterministic Constraint (Temperature)**: Beyond the text of the prompt, the model's internal parameter called temperature must be set to `0.0`. This turns off the creative randomness of the AI, ensuring the model delivers the exact same factual answer every time the query is run.
4. **Source Citation Enforcement**: The prompt must include a rule that obliges the model to append the source metadata identifier (like the chunk number) to every single technical claim or data list it writes. This turns the system from a black box into a transparent, verifiable tool for the operator.

## 3. System Implementation & Architecture

This section describes the practical implementation of the isolated RAG pipeline. The system is written in Python, leveraging **LangChain** for component orchestration, **ChromaDB** for vector storage, and **Ollama** for running the local language model. The implementation is contained within the main script file.

### 3.1 Data Ingestion and Processing Pipeline

The first phase of the architecture focuses on loading the document, extracting its textual content, and performing segment splitting (chunking).

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PDF_PATH = os.path.join(BASE_DIR, "manuale.pdf")
DB_DIR = os.path.join(BASE_DIR, "chroma_db")

print("--- 1. Caricamento PDF e Splitting ---")
loader = PyPDFLoader(PDF_PATH, extract_images=True)
documents = loader.load()
```

The script dynamically computes local paths using `os.path` to keep the application portable and isolated. The pipeline uses **PyPDFLoader** to parse the `manuale.pdf`. Crucially, the argument `extract_images=True` is enabled. This engineering choice guarantees that information contained inside technical tables, checklists, or visual schemas is extracted via background OCR, preventing critical data loss.

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=3000,
    chunk_overlap=700,
    length_function=len
)
chunks = text_splitter.split_documents(documents)

for i, chunk in enumerate(chunks):
    chunk.metadata["chunk_number"] = i + 1

print(f"Documento diviso in {len(chunks)} chunk.")
```

The loaded document is broken down using the **RecursiveCharacterTextSplitter**. The hyper-parameters are configured with a `chunk_size` of 3000 characters (approximately 500 words) and a high `chunk_overlap` of 700 characters. This massive overlap is intentionally set to avoid cutting data lists or specifications at page boundaries. Following the split, a custom `chunk_number` is injected into the metadata dictionary of each segment to provide absolute source tracciability.

### 3.2 Vector Storage and Semantic Search

Once text segments are generated, they are mapped into a dense multi-dimensional vector space and persisted locally.

```

print("\n-- 2. Generazione Embeddings (Modello Multilingue) --")
embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-m3",
    model_kwargs={'device': 'cpu'}
)

if os.path.exists(DB_DIR) and os.listdir(DB_DIR):
    print("Database vettoriale trovato in locale. Caricamento...")
    vector_store = Chroma(
        persist_directory=DB_DIR,
        embedding_function=embeddings
    )
else:
    print("Database non presente. Generazione embeddings in corso (potrebbe richiedere qualche minuto)...")
    vector_store = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=DB_DIR
    )
    print("Nuovo database vettoriale creato.")

```

The system implements the state-of-the-art **BAAI/bge-m3** multilingual model through HuggingFaceEmbeddings. This choice ensures highly accurate semantic matching for complex aviation terminology (e.g., Gimbal, RTH, Calibration) inside localized Italian sentences. The computation is locked to the host CPU ('device': 'cpu') to ensure seamless execution on any evaluation machine.

For database persistence, **ChromaDB** is configured with an automated caching check: if a compiled database index already exists in `chroma_db`, it is mounted instantly, optimizing execution times and bypassing redundant embedding generation.

### 3.3 Inference Core and Prompt Alignment

The logic core handles query execution, context assembly, and strict model constraint enforcement.

```

llm = OllamaLLM(model="llama3", temperature=0.0)

PROMPT_TEMPLATE = """Sei un ingegnere e assistente tecnico esperto e rigoroso. Rispondi alla domanda basandoti SOLO ed ESCLUSIVAMENTE sui
chunk di testo forniti nel contesto.

REGOLE TASSATIVE DI COMPORTAMENTO:
1. Distinzione dei componenti: Fai estrema attenzione a non confondere componenti o dispositivi diversi che condividono parole chiave simili nel
testo (es. batteria del drone vs batteria del radiocomando).
2. Dati certi e fattuali: Riporta solo i dati numerici, le specifiche e le istruzioni scritte esplicitamente.
Se noti cifre palesemente sballate o unite dall'OCR, prediligi i dati strutturati chiaramente negli altri chunk.
3. Integrità delle liste: Se nel contesto e' presente un elenco o una serie di punti protettivi, digitali tutti in ordine senza saltare o troncare nulla.
4. Cita le fonti: Alla fine di ogni blocco di dati o punto elenco, scrivi sempre il (Chunk Numero X) da cui proviene.

Se il contesto non e' sufficiente o e' del tutto assente, di': "Non trovo questa informazione nel manuale."

CONTESTO:
{context}

DOMANDA: {question}

RISPOSTA: """

prompt = PromptTemplate(template=PROMPT_TEMPLATE, input_variables=["context", "question"])

```

The inference layer relies on Ollama executing Llama 3. To eliminate creative deviations and ensure factual invariance, the model's temperature is locked to 0.0. The custom PromptTemplate acts as a behavioral alignment framework. It instructs the model to behave like a rigorous engineer, defining four safety rules: component

differentiation, factual dependency, list protection, and explicit source citation. A strict fallback clause forces a standardized refusal phrase when answers are missing from the context.

```
def query_rag(query_text):
    results = vector_store.similarity_search(query_text, k=3)

    if not results:
        return "Non trovo questa informazione nel manuale.", ["Nessun documento trovato nel database."]

    results = sorted(results, key=lambda x: x.metadata.get("chunk_number", 0))

    context_text = ""
    sources_info = []

    for doc in results:
        chunk_num = doc.metadata.get("chunk_number", "Sconosciuto")
        page_num = doc.metadata.get("page", "Sconosciuta")

        context_text += f"\n--- [Chunk Numero {chunk_num}] ---\n{doc.page_content}\n"
        sources_info.append(f"Chunk {chunk_num} (Pagina {page_num + 1})")

    full_prompt = prompt.format(context=context_text, question=query_text)
    response = llm.invoke(full_prompt)

    return response, sources_info
```

The `query_rag` function executes a similarity search retrieving exactly the top 3 most relevant chunks ( $k=3$ ). To solve narrative fragmentation, the code performs a metadata-driven sort operation using the `chunk_number` key. Sorting retrieved chunks chronologically restores the manual's native reading order before building the final context window.

## 4. Automated Testing & Evaluation Framework

To validate the deliverable, a dedicated test script was built. It assesses extraction precision and hallucination resistance by sending a fixed batch of 10 benchmark queries to the logical core.

```
import os
from rag_drone_manual import query_rag

# Definizione del set di 10 domande basate sul manuale del DJI Mini 2
test_questions = [
    # -----
    # 5 Domande PRESENTI nel manuale
    # -----
    "Quali sono le caratteristiche della batteria del drone?",
    "Quali sono i requisiti dell'ambiente di volo?",
    "Quali sono le tre modalita' di volo disponibili e dove si puo' passare dall'una all'altra?",
    "Quali sono le caratteristiche del radiocomando?",
    "Quali sono le quattro sottomodalita' previste per la funzione di volo intelligente QuickShot?",

    # -----
    # 5 Domande ASSENTI dal manuale (Test di rifiuto / Allucinazione)
    # -----
    "Come si cucina una pizza margherita?",
    "Come si sostituisce il motore elettrico del drone?",
    "Come si configura l'account DJI Fly usando un profilo Google o Apple?",
    "Qual e' la ricetta tradizionale per cucinare una carbonara perfetta?",
    "Quali sono le sanzioni previste se si vola sopra un'autostrada senza autorizzazione?"
]
```

The testing architecture isolates the RAG pipeline by importing the `query_rag` function as an independent module. The framework separates questions into two evaluation datasets:

- **In-Scope Queries (1-5):** Designed to check text mining and source extraction capabilities.
- **Out-of-Scope Queries (6-10):** Designed to evaluate boundary enforcement. This group challenges the model with unrelated general knowledge (cooking recipes) and adjacent drone topics deliberately missing from the manual (DIY motor repairs, legal traffic fines, third-party authentications).

```
def run_evaluation():
    output_filename = "report_test_dji.txt"
    print(f"Avvio del test di valutazione su {len(test_questions)} domande...")

    with open(output_filename, "w", encoding="utf-8") as file:
        file.write("=====\n")
        file.write("REPORT DI VERIFICA E TESTING - PIPELINE RAG (DJI Mini 2)\n")
        file.write("=====\n\n")

        for i, question in enumerate(test_questions, 1):
            section_type = "PRESENTI NEL MANUALE" if i <= 5 else "ASSENTI DAL MANUALE"

            print(f"[{i}/10] Elaborazione domanda ({section_type})...")

            # Richiamo al core logico del RAG
            answer, sources = query_rag(question)

            # Scrittura formattata dei risultati nel file di log
            file.write(f"Domanda {i} [{section_type}]:\n")
            file.write(f"-> {question}\n\n")
            file.write(f"Risposta LLM:\n{answer}\n\n")
            file.write(f"Chunk estratti dal DB: {' '.join(sources)}\n")
            file.write(f"-" * 60 + "\n\n")

    print(f"\nTest completato! I risultati sono stati salvati in '{output_filename}'.")
```

The evaluation harness loops through the questions, triggering vector searches and local model inference sequentially. All results are written directly to a structured, offline text ledger named `report_test_dji.txt`. This script creates a complete audit trail that lists the query, the raw extracted block indices, the target manual pages, and the final deterministic response generated by Llama 3.

## 5. Test Results & Evaluation Analysis

This section analyzes the performance of the local RAG pipeline based on the output file `report_test_dji.txt`. The test framework successfully evaluated all 10 benchmark questions, demonstrating optimal precision and strict alignment with the safety constraints.



## 5.1 In-Scope Query Evaluation

For the 5 queries regarding information present in the DJI Mini 2 manual, the logical core delivered precise, uncorrupted data.

- **Data Extraction Precision:** The system successfully extracted complete structural specifications from complex sections. For instance, when queried about the drone battery, the model correctly extracted multi-column parameters (such as capacity, voltage, and battery type) directly from the context chunks.
- **List Integrity:** In accordance with Behavior Rule number 3, lists regarding flight environment requirements and the four QuickShot intelligent sub-modes were generated completely in order, without shorthand summaries or truncations.
- **Source Verifiability:** Every factual claim generated by Llama 3 was followed by its correct source indicator, mapping back to the exact chunk number and manual page (e.g., Chunk 12 (Pagina 24)). This validates the mathematical sorting logic used to rebuild the manual's original flow before inference.

## 5.2 Out-of-Scope Query Evaluation & Hallucination Block

The remaining 5 queries tested the system's boundary enforcement against hallucinations. These questions presented the core with entirely unrelated topics (cooking recipes) or adjacent topics missing from the manual (third-party authentication, legal traffic fines, manual hardware replacement).

- **Zero-Shot Hallucination Mitigation:** In all 5 test cases, the system executed a complete and strict refusal. The output matched the fallback clause perfectly: "Non trovo questa informazione nel manuale."
- **Knowledge Isolation:** This result proves that setting the model's temperature to 0.0 completely turned off its creative randomness. Llama 3 was forced to ignore its general pre-trained parametric knowledge (e.g., how to make a pizza or a carbonara), acting purely as a rigid context processor.

## 6. Conclusion & Future Works

The developed prototype successfully delivers an isolated, independent, and secure RAG pipeline that operates entirely on local hardware (Edge Computing).

As a future development, the terminal-based command-line interface can be seamlessly transitioned to a web-based interactive interface using Streamlit. Because the pipeline is completely modular, the entire backend architecture—including the PyPDFLoader configuration, ChromaDB indexing, and the custom query functions—will remain unchanged, demonstrating the clean scalability of this software design.

